PRINCIPLES OF PROGRAMMING LANGUAGES FINAL 2025

NOT: BİREBİR SORULAR DEĞİL, ŞIKLAR VEYA DEĞİŞKENLER DEĞİŞEBİLİR, HAFIZADAN HAZIRLANDI HATA OLABİLİR.

True/False (1.5 points each)

1. In C, continue keyword will end the loop.
2. The operator associativity rules for expression evaluation define the order in which adjacent operators with the different precedence level are evaluated.
3. Unicode has more characters than ASCII.
4. All local variables are static in Java.
5. Functional side effects are when a function changes a two-way parameter or a local variable.
6. A dangling pointer is a pointer that points to memory that was never allocated.
7. In Python, range(2,10,2) gives the output 2,4,6,8.
8. In i++, i is incremented after it is used.
9. Haskell functions cannot take more than one argument.
10. Prolog variables must be defined with a type.

If false, explain why.

Multiple choice (4 points each)

1. What is a slice of an array?
    a. Substructure of an array
    b. ..
    c. ..
    d. ..
2. Which Prolog code should be used to check if two people like the same thing?
    a. likes(X,Z), likes(Y,Z), X\=Y.
    b. ..
    c. ..
    d. ..
3. What is the result of map *2 [1,2,3] in Haskell?
    a. [2, 4, 6]
    b. ..
    c. ..
    d. ..
4. Default parameters in Python?
    a. ..
    b. ..
    c. ..
    d. ..

PRINCIPLES OF PROGRAMMING LANGUAGES FINAL 2025

Klasik

1. Consider the following variables that are initialized:
   int x=5 int y=10 int z=1

   Do the following Boolean expressions result in short-circuited evaluation? Explain each.
   Hint: The logical-AND operator (&&) has higher precedence than the logical-OR operator (||).

   a. x==5 || y==10 || z==0
   b. x>10 && y<10
   c. x!=5 || y==10 && z==1
   d. ..
   e. ..

2. What are the values of r1 and r2, if the operands in the expressions are evaluated:
   a. left to right?
   b. right to left?

```
int update(int* x) {
        *x *= 2;
        return *x – 1;
}
void main() {
        int a=3, b=4, r1, r2;
        r1 = a + update(&a);
        r2 = update(&b) + b;
}
```

3. For each of the following parameter-passing methods, what is the output of the program? Assume static scope.
   a. Pass by value. 1 2 3 4 5 6 7 8 9
   b. Pass by name. 2 4 6 8 10

```
bool print_or_break(int x) {
    if (x < 10) {
            printf("%d ", x);
            return true;
    } else {
            return false;
    }
}

int f() {
    static int i = 0;
    i++;
}

void main() {
    while (print_or_break(f()));
}
```

4. For each of the following parameter-passing methods, what is the final version of array a[]? Assume static scope.
   a. Pass by reference. a a d a 4
   b. Pass by value-result. a a a a 4

```
int a[] = {'0', '1', '2', '3', '4'};
int i = 1;

void f(int x) {
        i++;
        x = 'a';
        a[2] = 'd';
        i--;
}

void main() {
        for (i = 0; i < 4; i++) {
                f(a[i]);
        }
}
```

5. How much memory is used? Assume 32-bit machine.

```
char d[][10] = {"Sunday",   "Monday", "Tuesday", "Wednesday",
                "Thursday", "Friday", "Saturday"}
```

S	U	N	D	A	Y	\			
M	O	N	D	A	Y	\			
T	U	E	S	D	A	Y	\		
W	E	D	N	E	S	D	A	Y	\
T	H	U	R	S	D	A	Y	\	
F	R	I	D	A	Y	\			
S	A	T	U	R	D	A	Y	\	

```
char *d[] = {"Sunday",   "Monday", "Tuesday", "Wednesday",
             "Thursday", "Friday", "Saturday"}
```

S	U	N	D	A	Y	\			
M	O	N	D	A	Y	\			
T	U	E	S	D	A	Y	\		
W	E	D	N	E	S	D	A	Y	\
T	H	U	R	S	D	A	Y	\	
F	R	I	D	A	Y	\			
S	A	T	U	R	D	A	Y	\	

6. What is the output of the code considering precedence rules?

```
void main() {
  int var1 = 5, var2 = 5;

  printf("%d \n", var1++);
  printf("%d \n", var1);
  printf("%d \n", var1++ * var1);
  printf("%d \n", var1);

  printf("%d \n", ++var2);
  printf("%d \n", var2);
  printf("%d \n", ++var2 * var2);
  printf("%d \n", var2);
}
```

C-Based Languages
postfix ++, --
prefix ++, --, unary +, -
*, /, %
binary +, -